



Instituto Tecnológico de Las Américas (ITLA)

Programación III

Proyecto Final

Sistema de Gestión Hotelera HotelSys RD

Nombre y Apellido: Raymel Enrique Guerrero
Reynoso

Matrícula: 2023-1677

Facilitador/a: Kelyn Tejada

Fecha: 16 de abril de 2026

Índice

Índice	2
1. Introducción	3
2. Estrategia de trabajo	3
2.1 Nombre del proyecto	3
2.2 Tecnología para aplicar	3
2.3 Objetivo del proyecto	3
2.4 Alcance del proyecto	3
2.5 Cronograma del proyecto	4
2.6 Definición del primer release	4
3. Metodología Scrum	5
3.1 Tareas a ejecutar	5
3.2 Equipo de trabajo	5
3.3 Herramientas y épicas	5
3.4 Ceremonias de Scrum	6
3.5 Historias de usuario	6
4. Plan de pruebas	8
4.1 Lista de requerimientos funcionales y no funcionales	8
4.2 Criterios de aceptación y rechazo de pruebas	9
4.3 Herramientas de pruebas	9
4.4 Cronograma de ejecución de pruebas	9
4.5 Plantilla para casos de prueba	9
4.6 Equipos de pruebas y responsabilidades	11
4.7 Plan de automatización de pruebas	11
4.8 Ejecución y demostración	11
5. Demostración y entregables	11
6. Conclusión	12
7. Bibliografía	12

1. Introducción

La administración hotelera requiere una gestión ordenada de habitaciones, clientes y reservaciones para reducir errores operativos y ofrecer una experiencia de servicio más consistente. En ese contexto, HotelSys RD se desarrolló como una aplicación web orientada a la gestión administrativa interna de un hotel.

El proyecto fue construido con C# y ASP.NET Core MVC, utilizando SQL Server y Entity Framework Core para persistencia de datos. Además del desarrollo funcional, el trabajo incorpora prácticas de planificación Agile-Scrum, control de versiones con GitHub, gestión del trabajo mediante Jira y un plan de pruebas con ejecución manual y automatizada.

Durante el desarrollo, el sistema evolucionó respecto a su propuesta inicial. La versión actual incluye autenticación, dashboard, gestión de habitaciones, clientes y reservaciones, reglas de negocio para disponibilidad, cálculo de costos, confirmación por correo electrónico y automatización de pruebas para flujos críticos.

2. Estrategia de trabajo

2.1 Nombre del proyecto

Sistema de Gestión Hotelera HotelSys RD.

2.2 Tecnología para aplicar

- Lenguaje de programación: C#.
- Framework web: ASP.NET Core MVC.
- Base de datos: SQL Server / LocalDB.
- ORM: Entity Framework Core.
- Interfaz: HTML, CSS y Bootstrap.
- IDE: Visual Studio.
- Control de versiones: Git y GitHub.
- Gestión del trabajo: Jira.
- Pruebas automatizadas: Selenium WebDriver con NUnit en C#.

2.3 Objetivo del proyecto

Desarrollar un sistema web de gestión hotelera que permita administrar habitaciones, clientes y reservaciones, con el fin de mejorar el control operativo del hotel, optimizar el registro de información y facilitar la consulta de datos de manera rápida, centralizada y organizada.

2.4 Alcance del proyecto

El sistema cubre la autenticación de acceso, la gestión CRUD de habitaciones, clientes y reservaciones, la visualización de métricas en un dashboard, la asignación automática del

número de habitación, el control de solapamiento de reservaciones, el cálculo del monto a pagar según noches de estadía y el envío de correos de confirmación de reservación.

La solución está orientada al uso administrativo interno del hotel y no incluye, en esta versión, pagos en línea, facturación electrónica, acceso para clientes externos ni reportes analíticos avanzados.

2.5 Cronograma del proyecto

Actividad	Fecha inicio	Fecha fin	Responsable
Definición del proyecto y herramientas	09/04/2026	09/04/2026	Raymel Guerrero
Documentación base del proyecto	10/04/2026	10/04/2026	Raymel Guerrero
Diseño de base de datos y módulo de habitaciones	11/04/2026	11/04/2026	Raymel Guerrero
Desarrollo del módulo de clientes	12/04/2026	12/04/2026	Raymel Guerrero
Desarrollo del módulo de reservaciones	13/04/2026	13/04/2026	Raymel Guerrero
Implementación de login, dashboard y reglas de negocio	14/04/2026	15/04/2026	Raymel Guerrero
Pruebas manuales, automatización y evidencias	15/04/2026	16/04/2026	Raymel Guerrero
Revisión final, video y entrega	16/04/2026	16/04/2026	Raymel Guerrero

2.6 Definición del primer release

El primer release de HotelSys RD incorpora los procesos esenciales para una operación administrativa básica del hotel. En esta versión, el usuario puede iniciar sesión, visualizar un dashboard general, registrar habitaciones, registrar clientes, crear reservaciones con fecha y hora, calcular el costo total de la estadía, evitar conflictos de disponibilidad y enviar un correo de confirmación con los detalles de la reserva.

Requerimientos funcionales del primer release:

- El sistema debe permitir iniciar sesión y cerrar sesión.
- El sistema debe mostrar un dashboard con métricas básicas del hotel.
- El sistema debe permitir registrar, listar, editar y eliminar habitaciones.

- El sistema debe asignar automáticamente el número de habitación en el alta.
- El sistema debe permitir registrar, listar, editar y eliminar clientes.
- El sistema debe permitir crear, listar, editar y eliminar reservaciones.
- El sistema debe manejar fecha y hora de entrada y salida.
- El sistema debe calcular cantidad de noches, precio por noche y total a pagar.
- El sistema debe impedir reservaciones solapadas para la misma habitación.
- El sistema debe enviar un correo de confirmación al crear una reservación.

Requerimientos no funcionales del primer release:

- La interfaz debe ser clara, visualmente organizada y fácil de usar.
- El sistema debe ejecutarse localmente sin errores durante la demostración.
- La información debe persistir en una base de datos relacional.
- El código debe mantenerse organizado y versionado en GitHub.
- Las pruebas automatizadas deben ejecutarse mediante Selenium WebDriver y NUnit.

3. Metodología Scrum

3.1 Tareas a ejecutar

- Levantamiento de requerimientos y definición del alcance.
- Diseño inicial de base de datos y entidades principales.
- Configuración del proyecto ASP.NET Core MVC y conexión a SQL Server.
- Desarrollo del módulo de habitaciones.
- Desarrollo del módulo de clientes.
- Desarrollo del módulo de reservaciones.
- Implementación de login y dashboard.
- Aplicación de reglas de negocio y notificaciones por correo.
- Diseño y ejecución de pruebas manuales.
- Desarrollo y ejecución de pruebas automatizadas.
- Documentación final, video y evidencias de entrega.

3.2 Equipo de trabajo

Por tratarse de un proyecto académico individual, una sola persona asume varios roles dentro del marco Scrum.

Rol	Responsable	Responsabilidad principal
Product Owner	Raymel Guerrero	Definir necesidades, alcance y prioridades del sistema.
Scrum Master	Raymel Guerrero	Planificar el trabajo, dar seguimiento al avance y remover bloqueos.

Developer	Raymel Guerrero	Implementar funcionalidades, reglas de negocio e integración técnica.
QA	Raymel Guerrero	Diseñar pruebas, ejecutar validaciones manuales y automatizadas.

3.3 Herramientas y épicas

- Visual Studio para el desarrollo y depuración.
- GitHub y GitHub Desktop para control de versiones.
- Jira para la gestión de historias, tareas y seguimiento.
- SQL Server / LocalDB para persistencia de datos.
- Selenium WebDriver y NUnit para pruebas automatizadas.

Épicas definidas para el proyecto:

- Autenticación del sistema.
- Gestión de habitaciones.
- Gestión de clientes.
- Gestión de reservaciones.
- Pruebas y documentación.

3.4 Ceremonias de Scrum

Ceremonia	Fecha	Objetivo
Sprint Planning	09/04/2026	Definir alcance, módulos, tareas y tecnología.
Daily Stand-up	10/04/2026 al 16/04/2026	Revisar avances, bloqueos y próximos pasos.
Sprint Review	16/04/2026	Presentar el incremento funcional del sistema.
Sprint Retrospective	16/04/2026	Identificar mejoras del proceso y aprendizajes del desarrollo.

3.5 Historias de usuario

A continuación se presentan diez historias de usuario con sus criterios de aceptación y puntos de historia, tal como exige la metodología definida para el proyecto.

ID	Historia de usuario	Criterios de aceptación	Puntos
HU-01	Como administrador, quiero iniciar sesión en el sistema para acceder a las funcionalidades administrativas.	1) El sistema debe permitir ingresar usuario y contraseña. 2) Si las credenciales son válidas, debe redirigir al dashboard. 3) Si son inválidas, debe mostrar un mensaje de error.	3
HU-02	Como administrador, quiero registrar habitaciones para llevar control de disponibilidad.	1) Debe permitir registrar tipo, precio y capacidad. 2) El número de habitación debe generarse automáticamente. 3) La habitación debe mostrarse en el listado después de guardarse.	5
HU-03	Como administrador, quiero editar habitaciones para mantener la información actualizada.	1) Debe permitir modificar los datos de una habitación existente. 2) Los cambios deben reflejarse en el listado. 3) No debe permitir datos inválidos.	3
HU-04	Como administrador, quiero registrar clientes para asociarlos a reservaciones.	1) Debe guardar nombre, apellido, correo, teléfono y documento. 2) Debe validar campos obligatorios. 3) El cliente registrado debe aparecer en el listado.	5
HU-05	Como administrador, quiero consultar clientes registrados	1) Debe listar todos los clientes. 2) Cada fila debe	2

ID	Historia de usuario	Criterios de aceptación	Puntos
	para localizarlos rápidamente.	mostrar datos esenciales. 3) Debe incluir acciones de detalle, edición y eliminación.	
HU-06	Como administrador, quiero crear reservaciones para asignar habitaciones a clientes.	1) Debe permitir seleccionar cliente y habitación. 2) Debe registrar fecha y hora de entrada y salida. 3) La reservación debe guardarse y verse en el listado.	5
HU-07	Como administrador, quiero que el sistema calcule el costo total de una reservación para informar el monto a pagar.	1) Debe calcular las noches de estadía. 2) Debe tomar el precio por noche de la habitación. 3) Debe mostrar el total a pagar.	5
HU-08	Como administrador, quiero impedir reservaciones solapadas para proteger la disponibilidad real de las habitaciones.	1) Si una habitación está ocupada en un rango, no debe permitir otra reserva solapada. 2) Debe mostrar mensaje de validación. 3) La reserva inválida no debe guardarse.	5
HU-09	Como administrador, quiero visualizar un dashboard con el resumen del sistema para consultar rápidamente el estado del hotel.	1) Debe mostrar el total de habitaciones, clientes y reservaciones. 2) Debe mostrar habitaciones disponibles, ocupadas y en mantenimiento. 3) Debe permitir	3

ID	Historia de usuario	Criterios de aceptación	Puntos
		navegar a los módulos principales.	
HU-10	Como administrador, quiero enviar una confirmación por correo al cliente cuando su reservación sea creada para mejorar la comunicación.	1) Debe enviarse un correo al crear la reservación. 2) El correo debe incluir habitación, entrada, salida y total a pagar. 3) Si el correo falla, la reservación debe seguir guardándose.	5

4. Plan de pruebas

4.1 Lista de requerimientos funcionales y no funcionales

Requerimientos funcionales:

- RF01. El sistema debe permitir iniciar sesión.
- RF02. El sistema debe permitir registrar, listar, editar y eliminar habitaciones.
- RF03. El sistema debe generar automáticamente el número de habitación.
- RF04. El sistema debe permitir registrar, listar, editar y eliminar clientes.
- RF05. El sistema debe permitir crear, listar, editar y eliminar reservaciones.
- RF06. El sistema debe manejar fecha y hora de entrada y salida.
- RF07. El sistema debe calcular precio por noche, cantidad de noches y total a pagar.
- RF08. El sistema debe impedir reservaciones solapadas.
- RF09. El sistema debe mostrar un dashboard con métricas básicas.
- RF10. El sistema debe enviar un correo de confirmación al crear una reservación.

Requerimientos no funcionales:

- RNF01. La interfaz debe ser clara, uniforme y fácil de usar.
- RNF02. El sistema debe ejecutarse correctamente en entorno local.
- RNF03. Los datos deben almacenarse en una base de datos relacional.
- RNF04. El código debe mantenerse organizado y versionado en GitHub.
- RNF05. Las pruebas automatizadas deben poder ejecutarse localmente con comandos documentados.

4.2 Criterios de aceptación y rechazo de pruebas

Criterios de aceptación:

- La funcionalidad evaluada ejecuta correctamente la operación esperada.
- Los datos se guardan correctamente en la base de datos.
- No se permiten datos inválidos en campos obligatorios.
- Las listas muestran la información registrada de forma coherente.
- Las reglas de negocio de reservaciones se cumplen sin errores.

Criterios de rechazo:

- La operación no persiste la información.
- El sistema presenta errores en creación, edición, eliminación o consulta.
- Se permiten datos inválidos o solapamientos no autorizados.
- La interfaz muestra páginas de error durante el flujo normal.
- Los resultados esperados no coinciden con la salida obtenida.

4.3 Herramientas de pruebas

Las pruebas del proyecto se apoyan en Visual Studio para ejecución y depuración, SQL Server / LocalDB para verificación de persistencia, navegador web para pruebas manuales, Selenium WebDriver con NUnit para automatización, GitHub para trazabilidad técnica y Jira para seguimiento funcional del trabajo.

4.4 Cronograma de ejecución de pruebas

Actividad de prueba	Fecha	Tipo de prueba	Responsable
Diseño de casos de prueba	14/04/2026	Planificación	Raymel Guerrero
Ejecución de pruebas de habitaciones	14/04/2026	Manual	Raymel Guerrero
Ejecución de pruebas de clientes	14/04/2026	Manual	Raymel Guerrero
Ejecución de pruebas de reservaciones	15/04/2026	Manual	Raymel Guerrero
Ejecución de automatización base	15/04/2026	Automatizada	Raymel Guerrero
Recolección de evidencias	15/04/2026	Evidencia	Raymel Guerrero
Revisión final de pruebas	16/04/2026	Cierre	Raymel Guerrero

4.5 Plantilla para casos de prueba

La siguiente plantilla se utilizó para documentar los casos de prueba funcionales del sistema.

ID	Módulo	Caso de prueba	Precondición	Pasos	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-01	Habitaciones	Registrar habitación válida	Sistema en ejecución	Ir a Habitaciones, pulsar Crear, completar formulario y guardar.	Tipo: Simple; Precio: 2500; Capacidad: 2	La habitación se guarda y se visualiza en el sistema.	Exitoso	Aprobado
CP-02	Clientes	Registrar cliente válido	Sistema en ejecución	Ir a Clientes, pulsar Crear, completar formulario y guardar.	Juan, Pérez, correo, teléfono y documentos.	El cliente se guarda y aparece en el listado.	Exitoso	Aprobado
CP-03	Reservaciones	Crear reservación válida	Cliente y habitación existentes	Ir a Reservaciones, completar selección de cliente, habitación y fechas, luego guardar.	Entrada y salida válidas, habitación disponible.	La reservación se crea y muestra el total calculado.	Exitoso	Aprobado
CP-04	Reservaciones	Validar fecha inválida	Cliente y habitación existentes	Crear reservación con salida menor o igual que la entrada.	Entrada: 16/04/2026; Salida: 15/04/2026	El sistema rechaza la operación y muestra error de validación.	Exitoso	Aprobado

4.6 Equipos de pruebas y responsabilidades

Debido a que el proyecto es individual, una sola persona asumió la planificación, ejecución manual, verificación de datos, automatización básica, captura de evidencias y corrección de defectos detectados durante la validación.

4.7 Plan de automatización de pruebas

La automatización se enfocó en flujos críticos y repetitivos del sistema utilizando Selenium WebDriver con NUnit en C#. Se organizó un proyecto de pruebas con carpetas Drivers, Pages, Tests y Utils para mantener un diseño modular cercano a Page Object Model.

- Flujos automatizados en la corrida demostrada: inicio de sesión, creación de clientes y creación de reservaciones.
- Estrategia de ejecución: levantar la aplicación con `dotnet run` y ejecutar la suite con `dotnet test HotelSysRD.AutomationTests --no-build` para evitar bloqueo del ejecutable.
- Objetivo principal: verificar que los flujos críticos del sistema pueden repetirse de manera consistente y documentada.

4.8 Ejecución y demostración

Como evidencia de la automatización se consideran: la estructura del proyecto `HotelSysRD.AutomationTests`, archivos de pruebas y page objects, ejecución de la suite automatizada y capturas de resultados satisfactorios. En la corrida final se obtuvieron casos exitosos para login, clientes y reservaciones.

5. Demostración y entregables

Para la entrega final se preparan los siguientes elementos:

- Video demostrativo mostrando el incremento funcional del primer release.
- Repositorio del proyecto con control de versiones en GitHub.
- Tablero o proyecto de Jira con épicas, historias, tareas y seguimiento.
- Código de pruebas automatizadas dentro del proyecto `HotelSysRD.AutomationTests`.
- Documento final en formato PDF con todos los puntos requeridos.

Entregable / enlace	Valor
Repositorio del proyecto	https://github.com/ElRay32/HotelSysRD
Jira / tablero de gestión	https://itla-team-yxlefcce.atlassian.net/jira/software/projects/SCRUM/boards/1
Código de pruebas automatizadas	https://github.com/ElRay32/HotelSysRD/tree/main/HotelSysRD.AutomationTests
Video demostrativo	https://www.youtube.com/watch?v=TZbwkIN_d9I

6. Conclusión

El desarrollo de HotelSys RD permitió aplicar de forma integrada conocimientos de programación orientada a objetos, desarrollo web con ASP.NET Core MVC, persistencia con Entity Framework Core y uso de una base de datos relacional. A nivel funcional, el sistema evolucionó desde un CRUD básico hasta una solución con autenticación, dashboard, reglas de negocio para disponibilidad, cálculo de costos y notificaciones por correo.

Desde la perspectiva metodológica, el proyecto también demuestra el uso de Agile-Scrum, la organización del trabajo en Jira, el control de versiones con GitHub y la planificación formal de pruebas. La incorporación de automatización con Selenium y NUnit refuerza la capacidad del sistema para ser validado de forma repetible en flujos críticos.

En conjunto, el proyecto cumple el propósito de evidenciar tanto la implementación técnica del software como la aplicación de buenas prácticas de ingeniería de software dentro de un contexto académico.

7. Bibliografía

1. Microsoft. (2026). ASP.NET Core documentation. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/>
2. Microsoft. (2024). Entity Framework Core overview. Microsoft Learn. <https://learn.microsoft.com/ef/core/>
3. Selenium Project. (2025). Selenium WebDriver documentation. <https://www.selenium.dev/documentation/webdriver/>
4. NUnit. (2026). NUnit Documentation Site. <https://docs.nunit.org/>
5. Atlassian. (2026). What is Jira Software? <https://www.atlassian.com/software/jira>
6. GitHub. (2026). GitHub Docs. <https://docs.github.com/>